Zero Breach Toolkit Enhanced

Security Assurance Candidate and Containment Hardening

I have completed an implementation and validation pass on the package. I cannot certify it as a security-sensitive production release, because the required independent testing, trusted publisher signature and native cross-platform evidence have not been obtained.

I have not substituted self-generated certificates or simulated test results for those requirements.

The original scanner and command-line interface have been preserved.

Modified Python source · New security tests · Containment diagnostics · Production gate · Deployment instructions · 919 KB

1. Implementation summary

Area	Implemented improvement	Remaining limitation
Windows containmen	Refuses execution if the VM has an active network adaptor; retains VM identity, administrator and firewall	Native Windows validation outstanding
macOS containmen	Rejects root execution; retains sandbox availability checks and deny-by-default profile	Relies on deprecated `sandbox-exec`; native
Child-process	Four IPv4/IPv6 endpoint and port combinations, tested from parent and isolated child processes	TCP only; cannot prove comprehensive network
Regression compatibili	Preserves previous diagnostic result keys	Broader system-level testing still required

The Windows improvement is particularly important: disconnecting the VM's network interfaces at the hypervisor provides a stronger network boundary than relying solely on guest firewall rules, assuming that the hypervisor is configured correctly.

2. Automated test results

544 Tests passed

21 Tests skipped

0 Tests failed

All 43 security tests passed. Python compilation and source-manifest verification also passed in the available Linux environment.

3. Command-line instructions

Extract the archive and enter the `Zero-Breach-Toolkit-main` directory.

Original interactive launch

```
python -X utf8 scan/zero-breach-toolkit.py
```

The original launch method remains available. Its embedded version string still identifies the v0.6.2 scanner, as this package is a containment hardening update.

Security tests

```
python -X utf8 security-test.py --suite security —verbose
```

Full functional and regression tests

```
python -X utf8 security-test.py --suite all --timeout 360
```

Source integrity and compilation

```
python -X utf8 tools/verify_manifest.py
python -X utf8 -m compileall -q scan tools
```

Windows — isolated disposable virtual machine

Run PowerShell as Administrator inside the VM, after disconnecting all VM network adaptors in your hypervisor:

```
powershell -NoProfile -ExecutionPolicy Bypass -File tools/
windows-vm-offline.ps1 --no-menu --no-banner —quick
```

The launcher refuses to proceed if the guest reports connected network adaptors or its required firewall checks fail.

Do not run this script on your normal Windows installation.

macOS — restricted sandbox

```
mkdir -p private-results
chmod 700 private-results

python3 -X utf8 tools/macos_contained.py \
  --output private-results —probe-only
```

If the containment probe passes:

```
python3 -X utf8 tools/macos_contained.py \
  --output private-results -- \
  --no-menu --no-banner —quick
```

For security-sensitive use, a network-disconnected virtual machine is preferable to relying on the deprecated macOS sandbox mechanism.

Child-process egress diagnostics

```
python -X utf8 tools/containment_probe.py --mode strict
```

Run this command inside the actual containment environment being assessed.

A passing result means the tested connections failed. It does not independently prove that the operating system prevented those connections.

4. Code status

Production certification: Not granted

The implementation work is complete for this hardening increment, but three major validation activities remain:

- Native execution and escape testing on supported Windows versions.
- Native macOS sandbox evaluation, including behaviour on supported macOS versions.
- Broader egress testing covering UDP, DNS, local proxies, non-Python child processes and attempted containment escapes.

The archive includes `CONTAINMENT-RELEASE.md`, describing the changes and remaining validation requirements.

5. Conclusion

The code is more restrictive and its automated Python tests pass. However, I cannot yet verify the Windows and macOS protections as production-grade security boundaries without executing the native tests on those platforms.